Black Box to Blueprint: Visualizing Leakage Propagation in Deep Learning Models for SCA

Suvadeep Hajra* and Debdeep Mukhopadhyay*
*Indian Institute of Technology Kharagpur, India

***Abstract*—Deep learning (DL)-based side-channel analysis (SCA) has emerged as a powerful approach for extracting secret information from cryptographic devices. However, its performance often deteriorates when targeting implementations protected by masking and desynchronization-based countermeasures, or when analyzing long side-channel traces. In earlier work, we proposed EstraNet, a Transformer Network (TN)-based model designed to address these challenges by capturing long-distance dependencies and incorporating shift-invariant attention mechanisms.**

In this work, we perform an in-depth analysis of the internal behavior of EstraNet and propose methods to further enhance its effectiveness. First, we introduce DL-ProVe (Deep Learning Leakage Propagation Vector Visualization), a novel technique for visualizing how leakage from secret shares in a masked implementation propagates and recombines into the unmasked secret through the layers of a DL model trained for SCA. We then apply DL-ProVe to EstraNet, providing the first detailed explanation of how leakage is accumulated and combined within such an architecture.

Our analysis reveals a critical limitation in EstraNet's multi-head GaussiP attention mechanism when applied to long traces. Based on this insights, we identify a new architectural hyperparameter which enables fine-grained control over the initialization of the attention heads. Our experimental results demonstrate that tuning this hyperparameter significantly improves EstraNet's performance on long traces (with upto $250K$ features), allowing it to reach the guessing entropy 1 using only 3 attack traces while the original EstraNet model fails to do so even with $5K$ traces.

These findings not only deepen our understanding of EstraNet's internal workings but also introduce a robust methodology for interpreting, diagnosing, and improving DL models for SCA.

***Index Terms*—SCA, Deep Learning, Shift-invariance, Transformer Network, Visualization**

I. Introduction

Side-Channel Analysis (SCA [1], [2]) represents a pervasive and practical threat to the security of physical cryptographic devices. By analyzing physical emanations such as power consumption or electromagnetic (EM) radiation, attackers can bypass the mathematical security of cryptographic algorithms and extract secret keys. In particular, profiling SCA [3], [4] leverages advanced machine learning models trained on data collected from a controlled but similar device to recover the secret key of a target device, posing a serious challenge to the security of modern digital infrastructure. As a result, evaluating the resilience of modern cryptographic devices against SCA is critical to ensuring the security of our daily digital interactions.

To mitigate the threat of SCA, security researchers have developed various countermeasures, including masking and desynchronization techniques. Masking countermeasures [5] split each sensitive intermediate cryptographic variable into multiple random shares, making information recovery dependent on modeling high-order statistical dependencies. Desynchronization-based countermeasures, such as clock jitter [6] and random delays [7], introduce misalignments in the traces, thereby hindering the accumulation of information across multiple side-channel observations or side-channel traces. These countermeasures make the evaluation of true resilience against SCA significantly more challenging. Moreover, long traces – common in software implementations of masked countermeasures – further complicate the analysis, as the relevant information is often concentrated in a few scattered features known as points of interest (PoIs). Efficiently extracting and leveraging these PoIs becomes increasingly difficult as trace lengths or attack window sizes grow, posing a serious obstacle to accurately assessing the true underlying security.

In recent years, Deep Learning (DL) has brought a paradigm shift in SCA by offering powerful tools that can automatically learn arbitrary function from raw, long traces and overcome certain countermeasures without requiring manual preprocessing [6], [8]–[11]. For instance, thanks to their ability to approximate complex functions, many DL models have successfully mounted attacks on implementations protected by masking countermeasures [9]–[12]. Convolutional Neural Networks (CNNs), due to their inherent shift-invariance, have proven highly effective against desynchronization-based countermeasures [6], [9], [13]. Recurrent Neural Networks (RNNs) [10] have demonstrated effectiveness against the combined challenges posed by masking and random delays. More recently, several works [12], [14] have highlighted the potential of Transformer Networks (TNs) for SCA. Their inherent ability to model long-distance dependencies makes them particularly well-suited for attacking long traces, where PoIs related to different secret shares may be long-distance apart. Furthermore, because TNs can be designed to exhibit shift-invariance, they can also be robustness against various desynchronization-based countermeasures. However, some of the existing models either fail to scale to long traces due to the quadratic computational complexity with respect to trace lengths or lack shift-invariance. Thus, achieving a combination of long-distance dependency modeling, shift-invariance, and scalability to long traces is a key challenge.

To address the above challenges, we previously introduced EstraNet [15], a TN-based model for SCA on long traces. While EstraNet demonstrated state-of-the-art perfor-

mance against various combinations of masking, random delay, and clock jitter countermeasures on traces up to $40K$ features long, its effectiveness, like that of many DL models, was primarily evaluated through final prediction accuracy. The internal mechanisms – how it precisely propagates and combines leakages from different secret shares to reconstruct the unmasked secret – remained a black box. This opacity is especially problematic for advanced models like EstraNet, which employ complex layers such as multi-head GaussiP attention, a key component responsible for its ability to capture long-distance dependencies. Without a clearer understanding of how the model internally processes information, systematic improvements beyond trial-and-error become difficult, limiting confidence in its robustness under varied conditions.

In this paper, we bridge the above gap by developing a methodology to dissect, understand, and enhance EstraNet. Our key contributions are as follows:

- We introduce **DL-ProVe** (**D**eep **L**earning leakage **Pro**pagation **Ve**ctor visualization), a novel and generalizable method to visualize the flow of information, or leakage, through the internal layers of a trained DL model. This technique offers clear insight into the role and mechanism of different layers during the processing of a trace.
- We apply DL-ProVe to EstraNet to conduct a detailed analysis of its internal behavior. This allows us to observe how the information of various secret shares propagates and combines to form the unmasked secret thorugh EstraNet when attacking masked implementations.
- We uncover a critical limitation of EstraNet when applied to long traces and propose an improvement by introducing a new hyperparameter that provides fine-grained control over the initialization of the model's attention heads. We empirically demonstrate that tuning this hyperparameter significantly enhances EstraNet's attack performance on long traces. Specifically, our experiments on a dataset with trace length of $250K$ features show that the improved EstraNet is able to reach the guessing entropy 1 using as few as 3 attack traces, whereas the vanilla EstraNet model fails to achieve the same even with $5K$ attack traces.

The remainder of this paper is organized as follows: Section II introduces the necessary background. In Section III, we provide an overview of the EstraNet model. Section IV presents DL-ProVe, a method for visualizing leakage propagation through deep learning model for SCA. In Section V, we apply DL-ProVe to analyze how leakages propagate through the layers of EstraNet. Section VI identifies a limitation of EstraNet on long traces and introduces a hyperparameter to improve its performance. Finally, Section VII summarizes the work and outlines potential directions for future research.

II. Background

In this section, we establish the notational conventions and present the necessary background.

A. Notations

For clarity, we define our notation as follows. A random variable, its specific value (instantiation), and its domain are represented by X, x, and $\mathcal{X}$, respectively. We use the notation $\mathbf{X}$ for the random vector and $\mathbf{x}$ for its instantiation. Matrices are denoted by Roman capital letters like X. We use $\mathbf{x}[i]$ for the i-th element of vector $\mathbf{x}$, and $M[i, j]$ for the entry in the i-th row and j-th column of matrix M. The probability distribution function is denoted by $\mathbb{P}[\cdot]$ and expectation is denoted by $\mathbb{E}[\cdot]$.

B. Side-Channel Analysis

The physical characteristics of a semiconductor device– such as its power consumption and electromagnetic (EM) emissions–are correlated with the internal data values it processes. SCA exploits this phenomenon to extract sensitive information from a cryptographic implementation, with the ultimate goal of revealing the secret key.

To perform SCA, an attacker first gains access to the target device, referred to as the Device Under Test (DUT). The attacker then repeatedly runs cryptographic operations with varying inputs on the DUT, recording the corresponding power or EM consumption during each execution. Each recording is known as a power or EM trace. The attacker then analyzes these traces using statistical or machine learning techniques to infer information about the secret key.

SCA can be broadly categorized into profiling and non-profiling attacks. Profiling SCA assumes that the attacker has access to an identical clone of the DUT. This enables the creation of a detailed a priori model—or profile—of the device's leakage behavior, which is later used to attack the actual DUT. In contrast, non-profiling SCA assumes that the attacker does not have access to a clone and must infer the secret key using only the traces collected from the target device itself.

This paper focuses exclusively on the profiling SCA setting.

C. Deep Learning-based Profiling SCA

Similar to other profiling SCAs, those based on Deep Learning (DL) consist of a profiling phase and an attack phase.

During the profiling phase, the attacker trains a DL model using a clone device with a known key. A large set of traces is collected from the clone device by running cryptographic operations on known inputs. For each trace, a corresponding intermediate variable of the cryptographic algorithm, $Z = F(X, K)$, is computed, where X is part of the plaintext/ciphertext, K is part of the key, and $F(\cdot, \cdot)$ is the cryptographic function. A DL model $f(\cdot; \theta)$ is then trained to take a trace $\mathbf{L}$ as input and output a probability distribution $\mathbf{p}$ over all possible values of Z. For a given trace $\mathbf{l}$, the model's output can be represented by $\mathbf{p} = f(\mathbf{l}; \theta^*)$ where $\mathbf{p} \in \mathbb{R}^{|\mathcal{Z}|}$, and $\mathbf{p}[j]$ denotes the predicted probability of $Z = j$.

In the subsequent attack phase, the attacker collects a set of trace-plaintext pairs, $(\tilde{\mathbf{l}}^i, \tilde{p}^i)_{i=0}^{T_a-1}$, from the actual target device. This data is used to evaluate every key guess $k \in \mathcal{K}$ using the trained model. The score assigned to each key guess is computed as:

$$\hat{\delta}_k = \sum_{i=0}^{T_a-1} \log \mathbf{p}^i[F(\tilde{p}^i, k)] \tag{1}$$

where $\mathbf{p}^i = f(\tilde{\mathbf{l}}^i; \theta^*)$. The final predicted key $\hat{k}$ is the one that maximizes this score: $\hat{k} = \text{argmax}_{k \in \mathcal{K}} \hat{\delta}_k$. The success of the attack is determined either by checking whether $\hat{k} = k^*$ (the correct key), or by computing the rank of the correct key among all candidates. The average rank of the correct key over multiple repetitions of the attack is a widely used metric known as guessing entropy.

D. Masking Countermeasures and Long-Distance Dependency

Masking [5], [16]–[18] is a prominent side-channel countermeasure designed to protect cryptographic devices by obfuscating the relationship between intermediate states of the cryptographic implementation and physical emanations such as power consumption. The technique works by splitting a sensitive intermediate variable, Z, into multiple random shares, $S_1, \ldots, S_d$, such that any proper subset of the shares remains independent of Z, while Z is recoverable by combining all of them (e.g., $Z = S_1 \oplus \cdots \oplus S_d$). These shares are then processed independently throughout the cryptographic algorithm. Consequently, the leakage corresponding to each share appears at different–often distant–locations in the side-channel trace. This scattering of related information introduces a complex analysis challenge known as long-distance dependency, where data points that are far apart in the time-series trace are interdependent. To overcome such a countermeasure, a DL model must be capable of identifying and combining these scattered leakage points to reconstruct information about the original secret Z [10]–[12], [14], [15].

E. Desynchronization-based Countermeasures and Shift-Invariance

Desynchronization-based countermeasures [7], [19], such as random delay, clock jitter, random delay interrupt, and shuffling, aim to prevent SCA by introducing random misalignments at the positions of the PoIs across traces. This makes it difficult to accumulate information from multiple traces during an attack. To effectively attack such countermeasures, DL models must be robust to these random shifts in the input traces–a property known as shift-invariance. More precisely, shift-invariance implies that the output of a DL model, i.e., $\mathbf{p} = f(\mathbf{l}; \theta^*)$, where $f(\cdot; \theta^*)$ denotes the function represented by the DL model with parameters θ^*, and $\mathbf{l}$ denotes the input trace, remains approximately the same when $\mathbf{l}$ is distorted by a random shift in either direction. DL models like Convolutional Neural Networks (CNNs) are inherently shift-invariant [6], [9]. In contrast, Transformer Networks (TNs) can be made shift-invariant through careful architectural design [12], [15].

F. Transformer Network (TN)

A TN is a kind of DL model composed of multiple transformer layers. The input trace is fed into the first transformer layer, and the output of each layer serves as the input for the subsequent layer. The final output of the last transformer layer is considered the output of the TN model.

Each transformer layer consists of a self-attention layer and a position-wise feed-forward layer. It takes a sequence $\mathrm{X} = [\mathbf{x}_0, \ldots, \mathbf{x}_{n-1}]^T \in \mathbb{R}^{n \times d}$ of n feature vectors as input and outputs another sequence $\mathrm{Y} = [\mathbf{y}_0, \ldots, \mathbf{y}_{n-1}]^T \in \mathbb{R}^{n \times d}$ where n is the sequence length (trace length in the context of SCA) and d is the dimension of the feature vectors. The output Y is given by

$$\hat{\mathrm{Y}} = f_{SA}(\mathrm{X}) + \mathrm{X}$$
$$\mathrm{Y} = f_{PFF}(\hat{\mathrm{Y}}) + \hat{\mathrm{Y}}$$

where $f_{SA} : \mathbb{R}^{n \times d} \mapsto \mathbb{R}^{n \times d}$ is the function representing the self-attention layer and $f_{PFF} : \mathbb{R}^{n \times d} \mapsto \mathbb{R}^{n \times d}$, is the function representing the position-wise feed-forward layer. The position-wise feed-forward layer independently applies a non-linear transformation to each feature vector, increasing the non-linearity (and, thus, its representation power) of the TN model. On the other hand, the self-attention layer captures the inter-dependencies among different features [12], [14], [15].

The self-attention layer is a key component of a TN. Different variations of this layer lead to vastly different types of TN models, each varying significantly in performance and computational cost. In the next part of this section, we briefly describe the self-attention layer and some of its variations, setting the groundwork for the following discussion.

1) Self-attention Layer: Let the sequence $\mathrm{X} = [\mathbf{x}_0, \ldots, \mathbf{x}_{n-1}]^T \in \mathbb{R}^{n \times d}$ of n feature vectors is the input of a self-attention layer and $\hat{\mathrm{Y}} = [\hat{\mathbf{y}}_0, \ldots, \hat{\mathbf{y}}_{n-1}]^T \in \mathbb{R}^{n \times d}$ be the corresponding output sequence. Then, in a self-attention layer, each output feature vector $\hat{\mathbf{y}}_i$, for $i = 0, \ldots, n-1$, is computed as

$$\hat{\mathbf{y}}_i = \sum_{j=0}^{n-1} \frac{k(f(\mathbf{x}_i), g(\mathbf{x}_j))}{\sum_{l=0}^{n-1} k(f(\mathbf{x}_i), g(\mathbf{x}_l))} \phi(\mathbf{x}_j) \tag{2}$$

where $f(\mathbf{x}_i), g(\mathbf{x}_i)$, and $\phi(\mathbf{x}_i)$ are some vector valued functions of input feature $\mathbf{x}_i$, and $k : \mathbb{R}^{d_k} \times \mathbb{R}^{d_k} \mapsto \mathbb{R}^+$ is called the kernel function. The kernel function takes a high score, $k(f(\mathbf{x}_i), g(\mathbf{x}_j))$, if the features $\mathbf{x}_i$, and $\mathbf{x}_j$ required to be combined and a low score otherwise. In summary, in self-attention, each output feature, $\hat{\mathbf{y}}_i$, is computed by taking the weighted sum of the input features $\phi(\mathbf{x}_0), \ldots, \phi(\mathbf{x}_{n-1})$ where the weight for the j-th input feature is given by the normalized score $k(f(\mathbf{x}_i), g(\mathbf{x}_j)) / \sum_{l=0}^{n-1} k(f(\mathbf{x}_i), g(\mathbf{x}_l))$. Therefore, if the i-th feature is required to be combined with the j-th feature, the weight of the j-th input feature in the i-th output feature can be significantly greater than weights of the other features, increasing the component of the j-th feature (in the input) in the i-th feature (in the output). In that way, self-attention layer can combine the information of the i and j-th features.

One of the most common kernel used in TN literature is obtained by setting the kernel as $k(f(\mathbf{x}_i), g(\mathbf{x}_j)) = \exp(\langle f(\mathbf{x}_i), g(\mathbf{x}_j) \rangle / \sqrt{d_k})$, where $\langle \cdot, \cdot \rangle$ represents the dot product between two vectors and d_k is the dimension of the outputs of $f(\cdot)$ and $g(\cdot)$. The above kernel leads to following self-attention known as softmax self-attention:

$$\hat{\mathbf{y}}_i = \sum_{j=0}^{n-1} \frac{\exp\left(\langle f(\mathbf{x}_i), g(\mathbf{x}_j)\rangle/\sqrt{d_k}\right)}{\sum_{l=0}^{n-1}\exp\left(\langle f(\mathbf{x}_i), g(\mathbf{x}_l)\rangle/\sqrt{d_k}\right)}\phi(\mathbf{x}_j) \tag{3}$$

$$= \sum_{j=0}^{n-1} \text{softmax}\left(\langle f(\mathbf{x}_i), g(\mathbf{x}_j)\rangle/\sqrt{d_k}\right)_j \phi(\mathbf{x}_j) \tag{4}$$

In the self-attention mechanism of the form of Eq. (2), the attention score from feature $\mathbf{x}_i$ to feature $\mathbf{x}_j$ does not depend on their positions, i or j, in the input sequence. Therefore, the actual order of the features in the input sequence does not affect the attention scores. In other words, permuting the input sequence also permutes the output sequence in the same way.

In many tasks, we want the attention a feature gives to another feature to depend on the distance between them. For instance, in the presence of random delay or clock jitter countermeasures, the distances between the informative features remain almost the same across all traces. These properties are exploited in the TN models proposed in [12], [15] by making the self-attention shift-invariant. This approach is described as follows:

Relative Positional Encoding: To make the self-attention shift-invariant, one can incorporate relative positional encoding into kernel function by generalizing it to $k_r(f(\mathbf{x}_i), g(\mathbf{x}_j), e(i-j))$, where $e(i-j)$ is a vector space encoding of the relative distance $i-j$. Therefore, the output of the resultant self-attention takes the form

$$\hat{\mathbf{y}}_i = \sum_{j=0}^{n-1} \frac{k_r(f(\mathbf{x}_i), g(\mathbf{x}_j), e(i-j))}{\sum_{l=0}^{n-1} k_r(f(\mathbf{x}_i), g(\mathbf{x}_l), e(i-l))}\phi(\mathbf{x}_j) \tag{5}$$

With relative positional encoding, the kernel function returns a high score not only based on the similarity between $\mathbf{x}_i$, and $\mathbf{x}_j$, but also considering their relative distance $i-j$.

III. EstraNet Model

The EstraNet model is a TN-based architecture specifically designed to meet three key requirements of SCA: (a) the ability to capture long-distance dependencies, thereby enhancing effectiveness against masking countermeasures; (b) linear computational complexity with respect to trace length, enabling scalability to long traces; and (c) shift-invariance, providing robustness against various desynchronization-based countermeasures such as random delay and clock jitter. In this section, we first provide a brief overview of EstraNet's overall architecture, followed by a discussion of its key component: the Multi-head GaussiP attention mechanism.

A. The Overall Architecture

The overall EstraNet architecture is shown in Figure 1a. As a Transformer Network (TN)-based model, EstraNet consists of multiple transformer-like layers, referred to as EstraNet layers, stacked sequentially. The input trace is first passed through two convolutional blocks, each followed by an average pooling layer, during which the trace is compressed to reduce its length. The output of the second convolutional block is then fed into the first EstraNet layer, whose output is passed to the next EstraNet layer, and so on.

The output sequence of the top EstraNet layer, denoted as $\mathbf{y}_0, \ldots, \mathbf{y}_{m-1}$ where each $\mathbf{y}_i \in \mathbb{R}^d$, is reduced to a single d-dimensional vector $\bar{\mathbf{y}}$ using a softmax attention layer. This vector $\bar{\mathbf{y}}$ is then passed to a classification layer for the final prediction.

Similar to a transformer layer, each EstraNet layer consists of a multi-head self-attention mechanism and a position-wise feed-forward network, as illustrated in Figure 1b. However, instead of standard self-attention, the EstraNet layer employs a novel attention mechanism called GaussiP attention. Moreover, the standard layer normalization operation used in vanilla transformer layers is replaced by a novel layer-centering operation in each EstraNet layer.

B. GaussiP Attention

The GaussiP attention is similar to self-attention with relative positional encoding given in Eq. (5). However, it uses a form of Gaussian kernel over the relative distance $i-j$ and special encoding to facilitate a) information flow from one feature to a distant feature, and b) controllable sparsity of the attention scores. Moreover, it uses a closed-form approximation for the denominator in Eq. (5). More precisely, the GaussiP attention's kernel takes the form:

$$\begin{aligned} k_{GPA}(i-j; s_p, c_p) &= \exp\left(-\frac{||\mathbf{v}_i - \mathbf{v}_j||_2^2}{2}\right) \\ &= \exp\left(-\frac{\beta_2^2 s_p^2 (i-j-c_p n)^2 ||\mathrm{W}_p\mathbf{p}||_2^2}{2}\right) \\ &= \exp\left(-\frac{\hat{\beta}_2^2 s_p^2 (i/n - j/n - c_p)^2 ||\mathrm{W}_p\mathbf{p}||_2^2}{2}\right), \end{aligned} \tag{6}$$

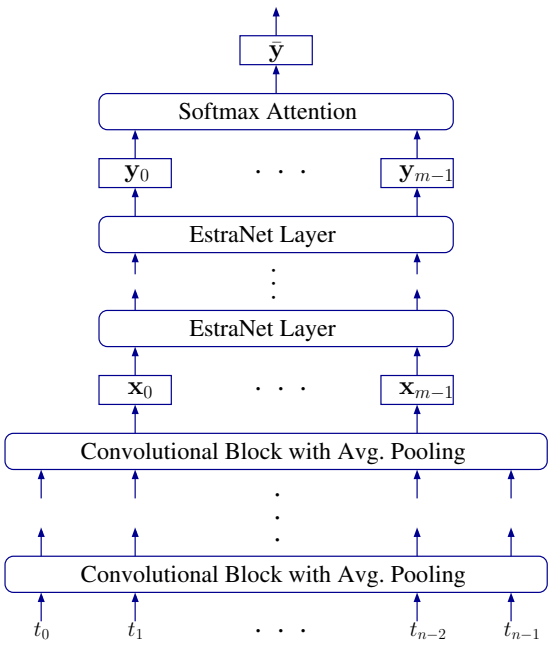
where $\mathbf{v}_i = \beta_2 s_p \mathrm{W}_p(\mathbf{b} + i\mathbf{p})$ and $\mathbf{v}_j = \beta_2 s_p \mathrm{W}_p(\mathbf{b} + j\mathbf{p} + c_p n\mathbf{p})$ be the vector space encoding of the positional indices i and j, with the attention scaling $s_p \in \mathbb{R}^+$, the attention center $c_p \in (0,1)$ being the parameters, $\mathrm{W}_p \in \mathbb{R}^{d_e \times d}$, $\mathbf{p} \in \mathbb{R}^d$ being two constants, and $\hat{\beta}_2 = n\beta_2 > 0$ being a hyperparameter of the model. To make the time and memory complexity of the attention linear in the sequence length n, the above kernel is approximately factorized using the d_p-dimensional Fourier feature map defined as

$$\begin{aligned} \phi_{\text{fr}}(\mathbf{x}) = \frac{1}{\sqrt{d_p}}[&\sin(\mathbf{w}_0^T\mathbf{x}), \ldots, \sin(\mathbf{w}_{d_p-1}^T\mathbf{x}), \\ &\cos(\mathbf{w}_0^T\mathbf{x}), \ldots, \cos(\mathbf{w}_{d_p-1}^T\mathbf{x})]^T, \end{aligned} \tag{7}$$

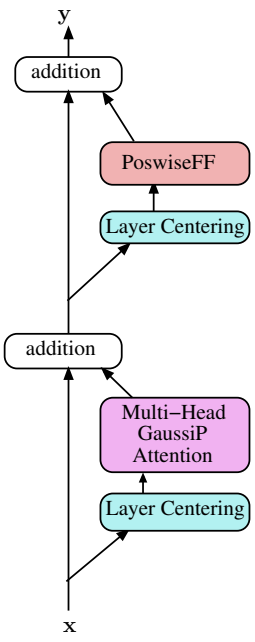
where $\mathbf{w}_0, \ldots, \mathbf{w}_{d_p-1}$ are the i.i.d. (independent and identically distributed) samples from d_e dimensional Gaussian distribution with zero mean and identity covariance matrix ensuring

$$k_{GPA}(i-j; s_p, c_p) \approx \phi_{\text{fr}}(\mathbf{i})^T \phi_{\text{fr}}(\mathbf{j}) \tag{8}$$

to hold. Furthermore, the GaussiP attention approximates the term $\sum_{l=0}^{n-1} k_{GPA}(i-l; s_p, c_p)$ in the closed form $n/\hat{\beta}_2 s_p ||\mathrm{W}_p\mathbf{p}||_2$. Therefore, the output feature vectors $\hat{\mathbf{y}}_i$s, of the resultant GaussiP attention,



(a) EstraNet Architecture.



(b) EstraNet Layer.

Fig. 1. The architecture of the EstraNet model and its layers.

$$\hat{\mathbf{y}}_i^T = \frac{\sum_{j=0}^{n-1} k_{GPA}\left(i-j; s_p, c_p\right) \phi(\mathbf{x}_j)^T}{\sum_{l=0}^{n-1} k_{GPA}\left(i-l; s_p, c_p\right)}, \text{following Eq. (5)}, \quad (9)$$

$$\approx \frac{\hat{\beta}_2 s_p ||\mathrm{W}_p \mathbf{p}||_2}{n} \sum_{j=0}^{n-1} \left(\phi_{\text{fr}}(\mathbf{i})^T \phi_{\text{fr}}(\mathbf{j})\right) \phi(\mathbf{x}_j)^T$$

$$\approx \frac{\hat{\beta}_2 s_p ||\mathrm{W}_p \mathbf{p}||_2}{n} \left[\phi_{\text{fr}}(\mathbf{i})^T \sum_{j=0}^{n-1} \phi_{\text{fr}}(\mathbf{j}) \phi(\mathbf{x}_j)^T\right], \quad (10)$$

for all $i = 0, \ldots, n-1$, can be computed in linear time and memory with respect to n.

Capturing Long-distance Dependency: In GaussiP attention, the i-th output feature puts high attention around $i - j - c_p n = 0$ or $j = i - c_p n$, allowing information flow from features near $i - c_p n$ to feature i. Since $i - c_p n$ can be far away from i (by initializing the attention center parameter $c_p \in (0, 1)$ properly), GaussiP attention allows information flow from one region of the traces to a distant region, enabling integration of the leakages of different regions of the traces.

C. Multi-head GaussiP Attention

Although the vanilla GaussiP attention facilitates information flow from features near $i - c_p n$ to feature i, this flow may be too restricted to effectively combine leakages from multiple regions. EstraNet solves this problem using multi-head GaussiP attention. More precisely, an H-head GaussiP attention, where H is a positive integer, uses H parallel GaussiP attentions to compute the output sequence. Therefore, the output of a H-head GaussiP attention is given by

$$\hat{\mathbf{y}}_i = \sum_{h=0}^{H-1} \mathrm{W}_o^{(h)} \hat{\mathbf{y}}_i^{(h)}, \quad \text{where}$$

$$\hat{\mathbf{y}}_i^{(h)} = \sum_{j=0}^{n-1} \frac{k_{GPA}\left(i-j; s_p^{(h)}, c_p^{(h)}\right)}{\sum_{l=0}^{n-1} k_{GPA}\left(i-l; s_p^{(h)}, c_p^{(h)}\right)} \phi(\mathbf{x}_j) \quad (11)$$

with $\hat{\mathbf{y}}_i^{(h)}$ being the i-th output feature and $s_p^{(h)} \in \mathbb{R}^+$ (the attention head scaling), $c_p^{(h)} \in (0, 1)$ (the attention head center), $\mathrm{W}_o^{(h)} \in \mathbb{R}^{d \times d_o}$ (the output projection weights) being three parameters of h-th head and $\phi(\cdot)$ being a vector valued function with output dimension d_o. The h-th head, for $h = 0, 1, \ldots, H-1$, of the attention propagates information from trace region near $\left(i - c_p^{(h)} n\right)$ to the i-th feature. Therefore, the H-head GaussiP attention facilitates information flow from H different regions, $\left(i - c_p^{(0)} n\right), \ldots, \left(i - c_p^{(H-1)} n\right)$, to the i-th feature. If any of these regions contain PoIs, information from those PoIs is combined near i-th output feature.

In [15], we demonstrated the effectiveness of EstraNet against masking countermeasures, even when combined with random delay or clock jitter countermeasures. However, due to its flexible and complex layer structure, how the information from different shares of a masked implementation is propagated and combined within an EstraNet model remains poorly understood. In the next section, we propose DL-ProVe, a method for visualizing the propagation of leakage from the various secret shares through a DL model. We will later apply this method to analyze leakage propagation in EstraNet.

IV. DL-ProVe: Deep Learning Leakage Propagation Vector Visualization

To visually track the propagation of leakages associated with different secret shares through the DL model, our approach involves estimating the informativeness (e.g., SNR) of each feature (where each feature can be a vector) at the output of an intermediate layer and plotting these values against feature indices, as shown in Figure 2. Such visualizations present a graphical representation of where information about

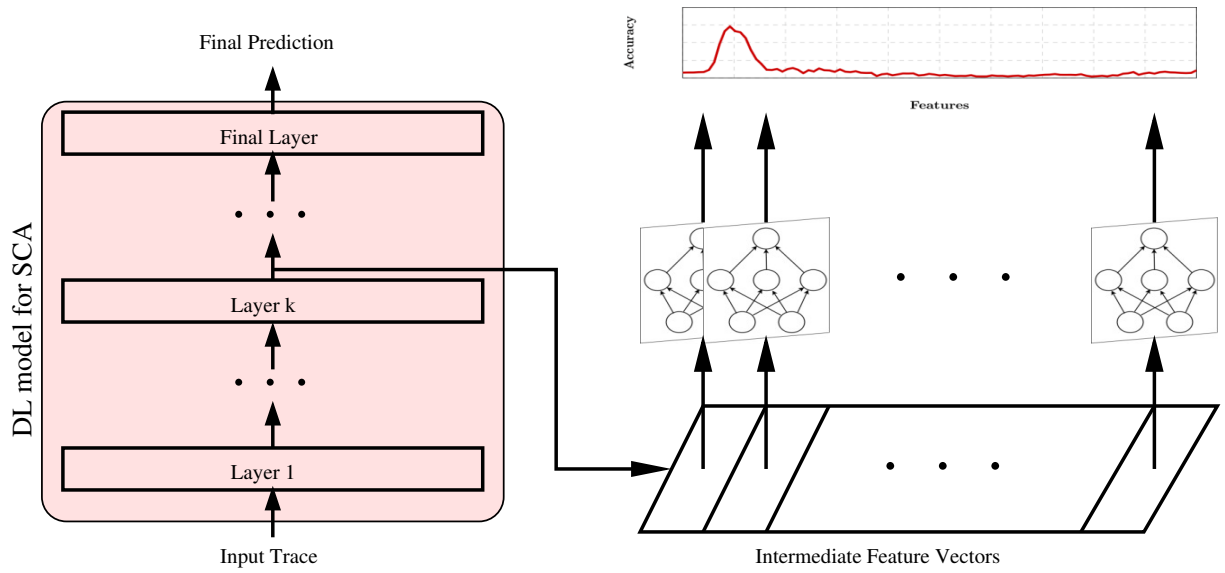


Fig. 2. Illustration of DL-ProVe method for visualizing the leakages at an Intermediate Layer of a DL model trained for SCA.

a secret cryptographic variable is located at the output of an intermediate layer of the DL model. By stacking these plots for each intermediate layer of the model, we can observe how information related to different secret cryptographic variables propagates through the network. An example of such a plot can be seen in Figure 5, which plots the propagation of leakages through a EstraNet model [15] on the ASCAD Random Key dataset[1].

A. Estimation of a Feature's Informativeness

Typically, the informativeness of a feature or a scalar variable with respect to another variable is evaluated using the SNR. The SNR of a cryptographic variable (e.g., a secret intermediate variable in the target cryptographic algorithm) at a feature of the side-channel traces is typically determined by the ratio of the signal component – estimated using the variance of the feature's component dependent on the cryptographic variable – to the noise component – estimated using the variance of the component independent of the variable. More precisely, if L is the random variable representing a feature and X represents the random variable corresponding to the cryptographic variable, the SNR of X at L is calculated as $Var(\mathbb{E}[L|X])/Var(L - \mathbb{E}[L|X])$.

When calculating the SNR for a cryptographic variable at a feature of a DL model's intermediate output, the above approach is not directly applicable. To elaborate, in a typical CNN, LSTM, or TN model, the output of an intermediate layer is a two-dimensional matrix. Let $\mathrm{Y} = [\mathbf{y}_0, \ldots, \mathbf{y}_{n-1}]^T \in \mathbb{R}^{n \times d}$ be the random variable corresponding to the output of the intermediate layer, where n is the length of the trace at that output. Note that n can be different, typically smaller, than the original length of the input trace, as the trace may undergo compression or expansion during the propagation through the intermediate layers of the DL model. The vectors $\mathbf{y}_0, \ldots, \mathbf{y}_{n-1} \in \mathbb{R}^d$ represent the features of the trace at the output of the intermediate layer, and d is the corresponding feature dimension. Therefore, each feature is a vector at the output of an intermediate layer of a DL model. Consequently, the above method for calculating the SNR, designed for scalar variables, cannot be directly applied to this case.

Since the conventional method of calculating SNR is not directly applicable to a feature vector, one straightforward approach to extending SNR calculation to the vector is to independently compute the SNR along each dimension of the vector and then combine them by averaging or taking the maximum. However, this simplistic method might yield an inaccurate estimate of informativeness as it does not leverage the interdependency among different dimensions of the feature vector. Consequently, we propose a DL-based method for estimating the informativeness of a feature vector with respect to a cryptographic variable.

Our proposed approach is grounded in the well-established fact that a DL model can effectively approximate any arbitrary function (with some loose restrictions) [20]. Leveraging this, we can estimate the informativeness of a feature vector $\mathbf{Y}$ with respect to a cryptographic variable X by employing a DL model to learn a function from the feature vector $\mathbf{Y}$ to the variable X. If $\mathbf{Y}$ is independent of X, training any DL models to predict X from $\mathbf{Y}$ will result in a prediction accuracy of the trained model similar to random guessing. Conversely, if $\mathbf{Y}$ contains substantial information about X, a properly trained DL model should predict X with significantly higher accuracy than random guessing when given $\mathbf{Y}$ as the input. Consequently, the accuracy obtained from the best-trained DL model to predict X can measure the informativeness of $\mathbf{Y}$ with respect to the variable X. A higher accuracy indicates higher informativeness.

We summarize the proposed method for estimating the informativeness as follows:

[1]https://github.com/ANSSI-FR/ASCAD/tree/master/ATMEGA_AES_v1/ATM_AES_v1_variable_key

1) If we want to estimate the informativeness of the i-th feature of an intermediate output of a DL model, collect a set of corresponding feature vector values $\mathcal{Y} = \{\mathbf{y}_{i,0}, \mathbf{y}_{i,1}, \ldots, \mathbf{y}_{i,T-1}\}$ by forward pass of T traces $\mathcal{L} = \{\mathbf{l}_0, \mathbf{l}_1, \ldots, \mathbf{l}_{T-1}\}$ through the DL model. Also, compute the values of the cryptographic variable X for each trace of $\mathcal{L}$. Let $\mathcal{X} = \{x_0, x_1, \ldots, x_{T-1}\}$ be the set of values of the variable.
2) Randomly split the set $\mathcal{D} = \{(\mathbf{y}_{i,0}, x_0), (\mathbf{y}_{i,1}, x_1), \ldots, (\mathbf{y}_{i,T-1}, x_{T-1})\}$ into train and test split, denoted as $\mathcal{D}^{train}$ and $\mathcal{D}^{test}$, respectively.
3) Train and tune a DL model on the train split $\mathcal{D}^{train}$ using the $\mathbf{y}_{i,j}$s as the input and x_js as the label.
4) Calculate the average accuracy of the DL model trained on the previous step on the test split $\mathcal{D}^{test}$. Report the average accuracy as the informativeness of the i-th feature about the variable X.

The method described above requires iterating through each feature of the intermediate layer. After computing accuracy values for all features, those can then be plotted against their respective feature indices to visualize the informativeness within the intermediate layer. Figure 2 provides an illustration of this approach.

In the following section, we employ the proposed approach to visualize how leakage propagates through a EstraNet model.

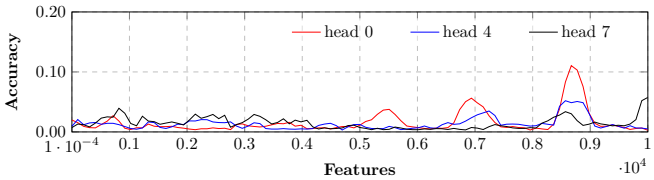
V. Visualizing the Leakage Propagation through EstraNet

As described in Section III, the key component of the EstraNet model is the multi-head GaussiP attention mechanism, which plays a central role in combining the leakages corresponding to different shares of a masked implementation to reconstruct the unmasked secret. In this section, we analyse and visualize the leakage propagation through EstraNet using DL-ProVe. Next subsection examines the propagation of leakages through the multi-head GaussiP attention layer and the first EstraNet layer. In the subsequent subsection, we illustrate how leakages propagate and combine through the entire EstraNet model. Before presenting the visualization results, we describe the dataset and the details of the corresponding cipher implementation.

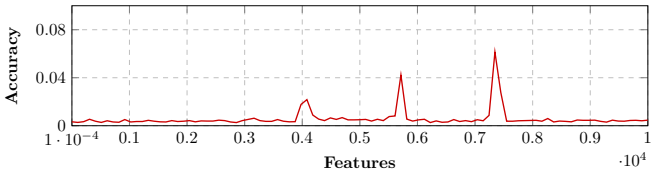
Details of the Dataset and Cipher Implementation: For the leakage visualization, we used the ASCAD Random Key dataset (ASCADr[2]). This dataset was collected from a first-order masked implementation of AES executed on an 8-bit ATMega8515 microcontroller [21]. For this dataset, most works in the literature focus on the output of the first round's third Sbox, i.e., $Sbox(p_3 \oplus k_3)$, where p_3 and k_3 are the third bytes of the plaintext and the first-round key, respectively.

Due to the first-order masking, the intermediate variable $Sbox(p_3 \oplus k_3)$ is never directly accessed during execution. Instead, it is processed using two independent secret shares: m_3 and $Sbox(p_3 \oplus k_3) \oplus m_3$, where m_3 is the third byte of a uniformly random mask variable. Consequently, the traces in the dataset do not contain direct leakage of the target variable

[2]https://github.com/ANSSI-FR/ASCAD/tree/master/ATMEGA_AES_v1/ATM_AES_v1_variable_key



(a) Prediction accuracy at the output of three different heads of the multi-head GaussiP attention of the first EstraNet layer.



(b) Prediction accuracy at the input of the multi-head GaussiP attention of the first EstraNet layer.

Fig. 3. Propagation of information related to the variable $Sbox(p_3 \oplus k_3) \oplus m_3$ through the different attention heads of the multi-head GaussiP attention of the first EstraNet layer.

$Sbox(p_3 \oplus k_3)$, but rather of the two shares: m_3 and $Sbox(p_3 \oplus k_3) \oplus m_3$. To extract information about $Sbox(p_3 \oplus k_3)$, EstraNet learns to combine the leakages corresponding to these two secret shares from the traces.

The traces in the dataset are $250K$ features long. For the analysis in this section, we selected an attack window of size $10K$ features, spanning from feature $78K$ to $88K$. An EstraNet model was trained on the profiling traces of the dataset using this $10K$-feature attack window, following the training procedure described in [15]. The trained model was then analyzed using the DL-ProVe method introduced in the previous section.

A. Leakage Propagation through the Multi-head GaussiP Attention

To understand leakage propagation through EstraNet, we first delve deeper into the propagation dynamics of its key component – the multi-head GaussiP attention layer – which enables the model to combine information from features that are far apart. The GaussiP attention facilitates two kinds of leakage propagation – forward and backward propagation.

1) Forward Propagation through GaussiP Attention: As discussed in Section III-C, a GaussiP attention layer primarily facilitates the propagation of leakage from the region around the $j = i - c_p n$-th feature of the input trace to the forward region around position i, where $c_p \in (0, 1)$ is a parameter of the layer and n denotes the input length. In a multi-head GaussiP attention layer with H heads, each head has its own parameter c_p, denoted as $c_p^{(0)}, \ldots, c_p^{(H-1)}$. As a result, leakages from positions around $\left(i - c_p^{(0)} n\right), \ldots, \left(i - c_p^{(H-1)} n\right)$ can propagate to the vicinity of the i-th feature. If some of these regions contain relevant leakages, their information is combined near position i, thereby enhancing the overall leakage strength.

In EstraNet, the values of $c_p^{(h)}$ in the first layer are initialized to be equally spaced within the range $(0, 0.5)$. More precisely, $c_p^{(h)}$ is initialized as $(1 + 2h)/4H$ for $h = 0, \ldots, H - 1$.

Although these parameters may shift during training, in practice they do not change drastically. As a result, lower-indexed heads propagate leakages from nearby regions, while higher-indexed heads propagate leakages from more distant past. This design enables EstraNet to combine leakage information from features that are either close together or far apart in input trace.

The forward propagation through multi-head GaussiP attention can be observed in Figure 3. The figure visualizes the propagation of leakages related to the cryptographic variable $Sbox(p_3 \oplus k_3) \oplus m_3$ through three different heads of the 8-head GaussiP attention in the first EstraNet layer using DL-ProVe (introduced in the previous section). The bottom figure, Figure 3b, shows the leakages at the input of the multi-head GaussiP attention, with three distinct PoIs near features $4K$, $5.8K$, and $7.4K$.

The top figure, Figure 3a, shows the leakages at the output of heads 0, 4, and 7. Head 0 propagates its input PoIs to positions near $5.5K$, $7K$, and $8.5K$. Head 4 propagates the PoI near $4K$ to approximately $7K$ and the PoI near $5.8K$ to around $8.5K$. Head 7 propagates the PoI the farthest, shifting the leakage near $4K$ to around $8.5K$. As a result, around position $7K$, the leakages of PoIs near $4K$ and $5.8K$ are superimposed. Similarly, near position $8.5K$, the leakages from all three PoIs are superimposed, leading to an enhanced signal at the output of the first EstraNet layer, as shown in Figure 4.

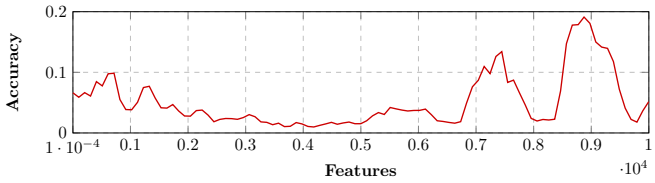
This figure illustrates how the leakages from the input of the first EstraNet layer propagate through all eight heads of the multi-head GaussiP attention and become amplified at the output. The bottom figure, Figure 4c, shows the leakages at the input of the multi-head GaussiP attention. The middle figure, Figure 4b, shows how the input PoIs propagate through the eight attention heads. Finally, the top figure, Figure 4a, illustrates how forward-propagated leakages are amplified on the right side due to the superimposition of leakages from multiple PoIs, facilitated by multiple attention heads.

2) Backward Propagation through GaussiP Attention: Apart from the forward leakage propagation discussed above, GaussiP attention also allows limited leakage propagation in the backward direction. More precisely, the Gaussian kernel (Eq. (6)) used in GaussiP attention is designed to permit leakage flow only in the forward direction, specifically from the region around $i - c_p n$ to i, since the attention weight $k_{\text{GPA}}(i-j; s_p, c_p)$ becomes negligibly small when $j \gg i - c_p n$. However, in practice, GaussiP attention approximates the kernel as $k_{\text{GPA}}(i-j; s_p, c_p) \approx \phi_{\text{fr}}(\mathbf{i})\phi_{\text{fr}}(\mathbf{j})$ (see Eq. (8)). This approximation causes the attention weights to remain nonzero even when $j \gg i - c_p n$, thus enabling backward leakage propagation.

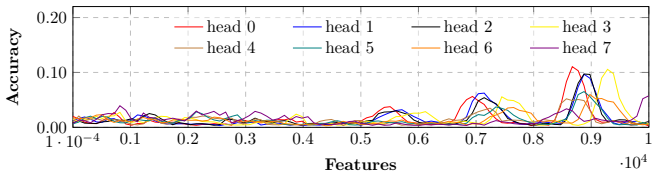
This backward leakage propagation across different heads is observable as smaller peaks in the left region (features 1–$4K$) of the individual head plots in Figure 4b. Furthermore, as illustrated in the left region of Figure 4a, the superposition of these small leakages leads to a significantly increased overall leakage strength.

B. Leakage Propagation through overall EstraNet Model

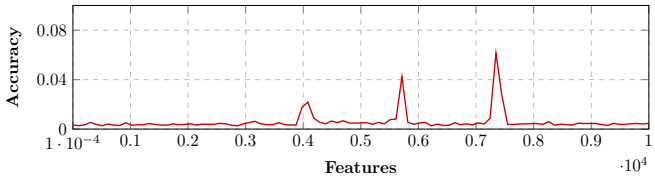
In this subsection, we illustrate the leakage propagation through the overall EstraNet architecture. In Figure 5, we plot



(a) Prediction accuracy at the output of the first EstraNet layer.

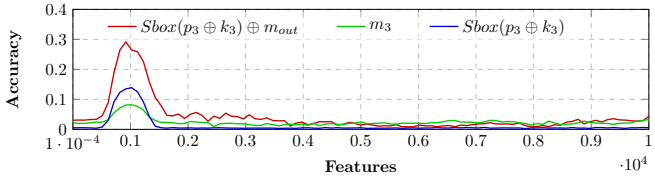


(b) Prediction accuracy at the output of the different heads of the multi-head GaussiP attention.

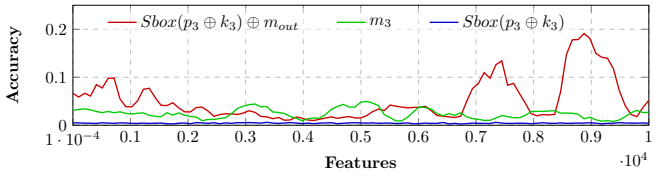


(c) Prediction accuracy at the input of the multi-head GaussiP attention of first EstraNet layer.

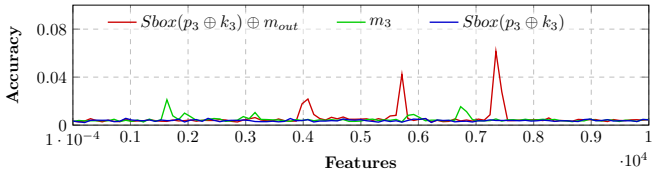
Fig. 4. Propagation of information related to the variable $Sbox(p_3 \oplus k_3) \oplus m_3$ through the different attention heads of the multi-head GaussiP attention.



(a) Prediction accuracy at the output of the second EstraNet layer.



(b) Prediction accuracy at the output of the first EstraNet layer.



(c) Prediction accuracy at the input of the first EstraNet layer.

Fig. 5. Propagation of leakages related to the intermediate variables: $Sbox(p_3 \oplus k_3) \oplus m_3$, m_3, and $Sbox(p_3 \oplus k_3)$ from the input of the first EstraNet layer to the output of the second EstraNet layer.

the informativeness of the two shares, m_3 and $Sbox(p_3 \oplus k_3) \oplus m_3$, as well as the unmasked secret $Sbox(p_3 \oplus k_3)$, at three intermediate layers of EstraNet using DL-ProVe. The bottom figure, Figure 5c, shows the accuracy at the input to the first EstraNet layer. The upper two figures, Figures 5b and 5a, plot the informativeness at the outputs of the first and second

EstraNet layers, respectively.

The bottom figure shows several prominent peaks for the two shares, indicating their significant leakage at the input of the first EstraNet layer. However, it does not show any peak corresponding to $Sbox(p_3 \oplus k_3)$, suggesting that information about the unmasked variable has not yet been reconstructed. In the middle figure, the leakages associated with the two secret shares are further propagated and enhanced, but no discernible information about the unmasked secret is present. In the top figure, the leakages of the two shares have been further propagated and superimposed near feature $1K$, resulting in the emergence of the unmasked secret $Sbox(p_3 \oplus k_3)$ – as indicated by the peak in the blue curve.

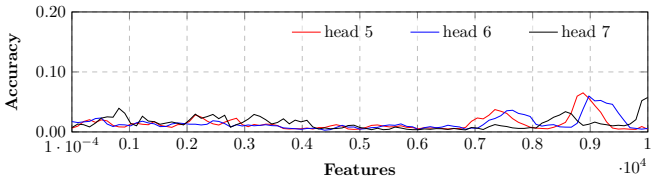
Finally, we aim to shed light on the nature of leakage propagation through the first and second layers of EstraNet. To this end, we examine the leakage plots at the outputs of these layers, as shown in Figures 5b and 5a. In Figure 5b, we observe that the leakages propagated in the forward direction exhibit significantly higher peaks than those in the backward direction, suggesting that forward propagation plays a more prominent role in the first EstraNet layer. Conversely, in Figure 5a, the leakage peaks appear primarily due to backward propagation, indicating that backward propagation has a more dominant influence in the second EstraNet layer.

In Appendix A, we visualize the leakage propagation through EstraNet on the CHES CTF 2020 SW3 dataset[3], which exhibits trends similar to those observed above. In the next section, we leverage these insights to enhance EstraNet's performance on long traces.

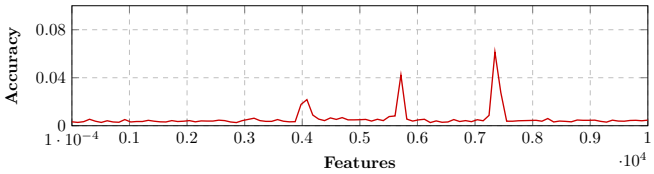
VI. Improving EstraNet's Performance on Long Traces

As explained in the previous section, the forward propagation of leakages through the multi-head GaussiP attention in the first EstraNet layer plays a crucial role in the model's performance. In the h-th head of the multi-head GaussiP attention of the first EstraNet layer, the forward propagation occurs by transmitting the leakage around feature $i - c_p^{(h)}n$ to the vicinity of feature i. Conversely, it can be interpreted as propagating leakage from feature j to feature $j + c_p^{(h)}n$, where the attention head center parameter $c_p^{(h)} \in (0, 0.5)$. In lower-indexed heads, $c_p^{(h)}$ takes a value close to 0, while higher-indexed heads are associated with values of $c_p^{(h)}$ closer to 0.5. As a result, a higher-indexed head tends to propagate leakage at feature j to a distant position at $j+c_p^{(h)}n$. Therefore, if a PoI at position j is located near the right boundary of the attack window (i.e., j is close to n), then the propagated position $j + c_p^{(h)}n$ may lie beyond the window. In such cases, the propagated leakage will fall outside the attack window and hence will have no positive effect on the model's performance.

This phenomenon is illustrated in Figure 6. The bottom figure shows the leakages of the secret share $Sbox(p_3 \oplus k_3) \oplus m_3$ at the input of the multi-head GaussiP attention layer in the first EstraNet layer. Three PoIs are clearly visible near features $4K$, $5.8K$, and $7.4K$. The upper figure shows the same variable's leakages at the output of heads 5, 6, and 7 of the 8-head GaussiP attention of the first EstraNet layer. It reveals that heads 5 and 6 successfully propagate the leakages of the first two PoIs but fail to propagate the last PoI within the attack window. Furthermore, head 7 is only able to propagate the leakage from the first PoI, with the remaining two lying beyond its effective range. As a result, these higher-indexed heads remain largely underutilized, potentially leading to suboptimal behavior of the model.



(a) Prediction accuracy at the output of last three heads of the multi-head GaussiP attention of the first EstraNet layer.



(b) Prediction accuracy at the input of the multi-head GaussiP attention of the first EstraNet layer.

Fig. 6. Propagation of information related to the variable $Sbox(p_3 \oplus k_3) \oplus m_3$ through the different attention heads of the multi-head GaussiP attention of the first EstraNet layer.

The above suboptimal behavior of multi-head GaussiP attention significantly impacts the model's performance on long traces or large attack windows. More precisely, since the attention head center $c_p^{(h)}$ of the h-th head is initialized as $(1 + 2h)n/4H$, the distance between two consecutive attention head centers is approximately $n/2H$ (though this may shift slightly during training). When H is kept constant, this distance increases proportionally with the size of the attention window n, making the superposition of multiple propagated PoIs by different attention heads increasingly difficult. Moreover, if the PoIs are located near the right boundary of the attack window, superposition becomes even more difficult, as the PoIs propagated by higher-indexed heads may fall outside the window (as illustrated in Figure 6). As a result, EstraNet's performance degrades sharply with increasing attack window size. To verify this observation, we conducted experiments using EstraNet with varying attack window sizes. Table I reports T_{GE1}, the number of attack traces required for EstraNet to reach the guessing entropy 1, on three different attack window sizes: $10K$, $40K$, and $250K$ (full length) using the ASCADr dataset (refer to Section V for dataset details). Each experiment was repeated three times, and the table reports the results from all three training runs. The results indicate that EstraNet maintains stable performance for attack window sizes up to $40K$ features. However, for attacks on full-length traces (with $250K$ feature), EstraNet fails to reach the guessing entropy 1 even using $5K$ attack traces in all three runs.

[3]https://ctf.spook.dev/

TABLE I
THE NUMBER OF ATTACK TRACES REQUIRED BY ESTRANET TO REACH GUESSING ENTROPY 1 (T_{GE1}) ON THE ASCADR DATASET WHILE THE ATTACKS ARE CONDUCTED WITH DIFFERENT ATTACK WINDOW SIZES.

Attack	Training Runs		
Winsow Size	run 1	run 2	run 3
$10K$	4	6	5
$40K$	8	7	7
$250K$	$> 5K$	$> 5K$	$> 5K$

A. Improving the Initialization of the Multi-head GaussiP Attention's Head Centers

One way to improve the performance of EstraNet on long traces is by increasing the number of attention heads H. A larger H enables denser information flow, making the superimposition of leakages from multiple PoIs more feasible. However, increasing H also raises the number of parameters and the computational cost, potentially leading to overfitting and slower training. Instead, we propose introducing a new hyperparameter – the maximum attention center, denoted as $\mu_c \in (0, 1)$ – to control the range of attention head centers $c_p^{(h)}$ in the first EstraNet layer. Specifically, the attention centers are initialized to be equally spaced within the range $(0, \mu_c)$. That is, each $c_p^{(h)}$ is initialized as:

$$c_p^{(h)} = \frac{1 + 4h\mu_c}{4H}$$

where $h = 0, \ldots, H-1$. Note that setting $\mu_c = 0.5$ recovers the original initialization used in the vanilla EstraNet model.

To evaluate the impact of μ_c on EstraNet's performance over long traces (or large attack windows), we conducted experiments using full-length traces from the ASCADr dataset (i.e., using attack window size of $250K$ features) while setting μ_c to 0.25 and 0.15, in contrast to the default value of 0.5. As noted earlier in Table I, with the default $\mu_c = 0.5$, EstraNet fails to reach guessing entropy 1 within $5K$ attack traces in any of the three training runs. However, as shown in Table II, reducing μ_c significantly improves EstraNet's performance. The best results are observed with $\mu_c = 0.15$, where two out of three training runs reach guessing entropy 1 using fewer than 4 attack traces. In the remaining run, EstraNet reaches the guessing entropy 1 using 122 attack traces. The results clearly underscore the advantage of the introduced hyperparameter.

TABLE II
THE NUMBER OF ATTACK TRACES REQUIRED BY ESTRANET TO REACH GUESSING ENTROPY 1 (T_{GE1}) ON THE ASCADR DATASET FOR DIFFERENT VALUES OF μ_c ON ATTACK WINDOW SIZE OF $250K$ FEATURES.

μ_c	Training Runs		
	run 1	run 2	run 3
0.5 (vanilla)	$> 5K$	$> 5K$	$> 5K$
0.25	5	2453	32
0.15	122	3	4

VII. CONCLUSION

In this work, we presented a comprehensive analysis of EstraNet, a TN-based model for DL-based SCA on long traces. While EstraNet was previously shown to be effective against a wide range of countermeasures, its internal behavior remained largely unexplored. To bridge this interpretability gap, we introduced **DL-ProVe** – a novel and general-purpose visualization technique for tracing the propagation and recombination of leakage through the internal layers of DL models.

Applying DL-ProVe to EstraNet allowed us to uncover how leakages from masked secret shares evolve across layers, offering the first detailed insights into the model's leakage propagation and aggregation mechanisms. Through this analysis, we identified a critical limitation of EstraNet's multi-head GaussiP attention when applied to long traces. In response, we proposed a new architectural hyperparameter to enable fine-grained control over the initialization of the attention heads. Our experimental results with traces of lengths upto $250K$ features demonstrated that this modification substantially improves EstraNet's performance, reducing the number of required attack traces to reach the guessing entropy 1 from more than $5K$ to as few as 3 traces.

Beyond EstraNet, our findings highlight the broader potential of DL-ProVe as a tool for understanding, diagnosing, and enhancing the internal behavior of DL models in the SCA domain. Future work could extend DL-ProVe to other architectures and explore its use for automated model refinement and countermeasure assessment.

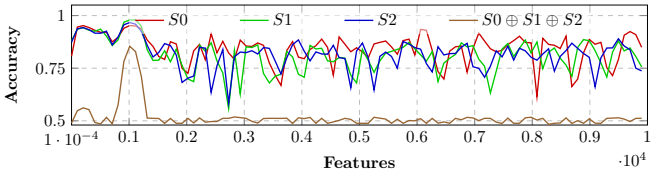
APPENDIX A
LEAKAGE PROPAGATION THROUGH ESTRANET ON THE CHES CTF 2020 SW3 DATASET

In this section, we investigate the propagation of leakages through EstraNet on the CHES CTF 2020 SW3 (CHES20) dataset[4]. The traces were collected from a second-order masked implementation of the Clyde-128 [22] tweakable block cipher running on an ARM Cortex-M0 32-bit microcontroller. Each trace in the dataset is $62.5K$ features long. We selected an attack window of size $10K$ features, ranging from $46K$ to $56K$. Clyde-128 follows an LS design and operates on a (4×32)-bit state, where 4 corresponds to the size of the non-linear S-box and 32 to the size of the linear L-box. Our target is the state bit located at the 1^{st} row and the 17^{th} column (from the right) of the first-round S-box operation (following [15]).

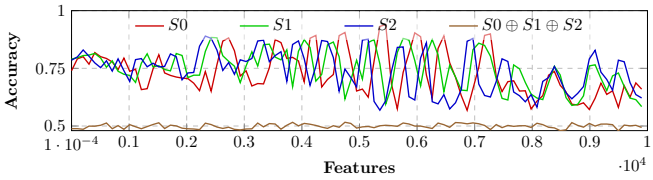
The plots of leakage propagation using DL-ProVe are shown in Figure 7. The figure illustrates the propagation of leakage for the three shares of the target bit, denoted as $S0$, $S1$, and $S2$, as well as their recombination into the unmasked target bit $S0 \oplus S1 \oplus S2$, across two layers of EstraNet. The bottom plot, Figure 7c, shows the leakage of the four variables at the input of the first EstraNet layer. The middle and top plots – Figures 7b and 7a, respectively – depict the output of the first and second EstraNet layers.

By examining the plots, we observe trends similar to those in the ASCADr dataset. The first EstraNet layer propagates the leakage of each individual share across multiple positions, which become amplified through superposition. However, the reconstruction of the target bit does not occur at the output of this layer. The second EstraNet layer further propagates and

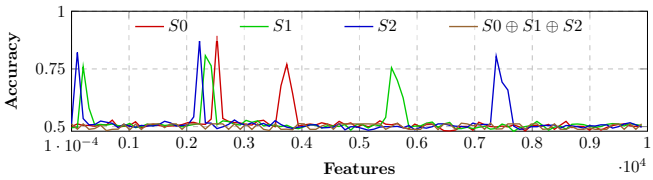
[4]https://ctf.spook.dev/



(a) Prediction accuracy at the output of the second EstraNet layer.



(b) Prediction accuracy at the output of the first EstraNet layer.



(c) Prediction accuracy at the input of the first EstraNet layer.

Fig. 7. Propagation of leakages related to three seceret shares, denoted as $S0$, $S1$, and $S2$ from the input of the first EstraNet layer to the output of the second EstraNet layer on the CHES CTF 2020 (SW3) dataset. We considered only a single bits of the shares. Therefore, the accuracy of random guessing is 0.5. Informative features shows significantly higher accuracy than 0.5.

amplifies the leakage from all three shares. These leakages then superimpose around feature index $1K$, enabling the reconstruction of the target bit $S0 \oplus S1 \oplus S2$, as indicated by the peak in the brown curve.

These results further underscore the consistency of EstraNet's internal mechanisms across multiple datasets.

References

[1] P. C. Kocher, "Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems," in *CRYPTO '96, California, USA,*, ser. LNCS, N. Koblitz, Ed., vol. 1109, 1996, pp. 104–113.

[2] P. C. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *CRYPTO, USA, 1999*, ser. LNCS, vol. 1666. Springer, 1999, pp. 388–397.

[3] S. Chari, J. R. Rao, and P. Rohatgi, "Template attacks," in *CHES, USA, 2002*, ser. LNCS, B. S. K. Jr., Ç. K. Koç, and C. Paar, Eds., vol. 2523. Springer, 2002, pp. 13–28.

[4] W. Schindler, K. Lemke, and C. Paar, "A stochastic model for differential side channel cryptanalysis," in *CHES, UK, 2005*, ser. LNCS, J. R. Rao and B. Sunar, Eds., vol. 3659. Springer, 2005, pp. 30–46.

[5] S. Chari, C. S. Jutla, J. R. Rao, and P. Rohatgi, "Towards sound approaches to counteract power-analysis attacks," in *CRYPTO, USA, 1999*, ser. LNCS, vol. 1666. Springer, 1999, pp. 398–412.

[6] E. Cagli, C. Dumas, and E. Prouff, "Convolutional neural networks with data augmentation against jitter-based countermeasures - profiling attacks without pre-processing," in *CHES, Taiwan, 2017*, ser. LNCS, vol. 10529. Springer, 2017, pp. 45–68.

[7] J. Coron and I. Kizhvatov, "An efficient method for random delay generation in embedded software," in *CHES, Switzerland, 2009*, ser. LNCS, vol. 5747. Springer, 2009, pp. 156–170.

[8] H. Maghrebi, T. Portigliatti, and E. Prouff, "Breaking cryptographic implementations using deep learning techniques," in *SPACE, India, 2016*, ser. LNCS, vol. 10076. Springer, 2016, pp. 3–26.

[9] G. Zaid, L. Bossuet, A. Habrard, and A. Venelli, "Methodology for efficient CNN architectures in profiling attacks," *TCHES*, vol. 2020, no. 1, pp. 1–36, 2020.

[10] X. Lu, C. Zhang, P. Cao, D. Gu, and H. Lu, "Pay attention to raw traces: A deep learning architecture for end-to-end profiling attacks," *TCHES*, vol. 2021, no. 3, pp. 235–274, 2021.

[11] G. Perin, L. Wu, and S. Picek, "Exploring feature selection scenarios for deep learning-based side-channel analysis," *Cryptology ePrint Archive*, 2021.

[12] S. Hajra, S. Saha, M. Alam, and D. Mukhopadhyay, "TransNet: Shift invariant transformer network for side channel analysis," in *AFRICACRYPT 2022, Fes, Morocco, 2022, Proceedings*, ser. LNCS, L. Batina and J. Daemen, Eds. Springer Nature Switzerland, 2022, pp. 371–396.

[13] L. Masure, N. Belleville, E. Cagli, M. Cornelie, D. Couroussé, C. Dumas, and L. Maingault, "Deep learning side-channel analysis on large-scale traces - A case study on a polymorphic AES," in *ESORICS, UK, 2020*, ser. LNCS, vol. 12308. Springer, 2020, pp. 440–460.

[14] E. Bursztein, L. Invernizzi, K. Král, D. Moghimi, J. M. Picod, and M. Zhang, "Generic attacks against cryptographic hardware through long-range deep learning," *CoRR*, vol. abs/2306.07249, 2023.

[15] S. Hajra, S. Chowdhury, and D. Mukhopadhyay, "EstraNet: An efficient shift-invariant transformer network for side-channel analysis," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2024, no. 1, p. 336–374, 2023.

[16] T. S. Messerges, "Securing the AES finalists against power analysis attacks," in *FSE, USA, 2000*, ser. LNCS, vol. 1978. Springer, 2000, pp. 150–164.

[17] J. Coron and L. Goubin, "On boolean and arithmetic masking against differential power analysis," in *CHES, USA, 2000*, ser. LNCS, vol. 1965. Springer, 2000, pp. 231–237.

[18] M. Akkar and C. Giraud, "An implementation of DES and AES, secure against some attacks," in *CHES, France, 2001*, ser. LNCS, vol. 2162, no. Generators. Springer, 2001, pp. 309–318.

[19] J.-S. Coron and I. Kizhvatov, "Analysis and improvement of the random delay countermeasure of CHES 2009," in *CHES, USA, 2010*, ser. LNCS, vol. 6225. Springer, 2010, pp. 95–109.

[20] Z. Lu, H. Pu, F. Wang, Z. Hu, and L. Wang, "The expressive power of neural networks: A view from the width," in *NeurIPS, Long Beach, CA, USA*, I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, Eds., 2017, pp. 6231–6239.

[21] R. Benadjila, E. Prouff, R. Strullu, E. Cagli, and C. Dumas, "Deep learning for side-channel analysis and introduction to ASCAD database," *J. Cryptogr. Eng.*, vol. 10, no. 2, pp. 163–188, 2020.

[22] D. Bellizia, F. Berti, O. Bronchain, G. Cassiers, S. Duval, C. Guo, G. Leander, G. Leurent, I. Levi, C. Momin, O. Pereira, T. Peters, F. Standaert, B. Udvarhelyi, and F. Wiemer, "Spook: Sponge-based leakage-resistant authenticated encryption with a masked tweakable block cipher," *IACR Trans. Symmetric Cryptol.*, vol. 2020, no. S1, pp. 295–349, 2020.